Synchronisation des processus

Prof Ahmed EL OIRRAK
Dépt D'Informatique, SMI S4
Faculté des sciences Semlalia

Exclusion mutuelle avec attente active

- **désactivation des interruptions**
- On peut penser à désactiver les interruptions : une nouvelle interruption n'aura pas de conséquence sur le processus courant. Mais cela est très dangereux :

cela gêne considérablement le système d'exploitation

le processus courant peut, pour diverses raisons, ne pas réactiver les interruptions

De plus sur un système à plusieurs processeurs le dispositif ne marche pas. Conclusion : ce n'est pas la bonne solution !

variables "verrous"

- Soit une variable globale Lock, correspondant à une ressource donnée et initialisée à 0. Supposons qu'un processus désire entrer dans sa section critique, c'est à dire désire utiliser la ressource associée à Lock :

si Lock = 0, alors le processus met Lock à 1 et entre dans sa section critique ; après utilisation de la ressource, le processus met Lock à 0 et sort de sa section critique.

si Lock = 1, le processus attend que Lock soit remise à 0 (attente active)

- Mais difficulté : entre le moment où le processus lit Lock et le moment où il écrit dans Lock (lecture et écriture étant deux opérations distinctes successives), un autre processus peut intervenir !

Conclusion : ce n'est pas la bonne solution !

Alternance stricte

- On définit une variable partagée entière Turn, initialisée à 0 et globale.
- Considérons 2 processus : le processus P0 et le processus P1. Chacun des deux processus ne peut entrer

dans sa section critique que si la valeur de Turn est égale à son numéro (0 ou 1).

Supposons que le processus P0 lise Turn et constate que sa valeur est 0 ; il entre dans sa section critique.

Si le processus P1 lit à son tour Turn (valeur 0), il doit attendre dans une boucle le passage de Turn à 1

(attente active). Quand le processus P0 sort de sa section critique, il met Turn à 1. Le processus P1 peut

alors entrer dans sa section critique; quand il en sortira, il mettra Turn à 0. Cette méthode correspond aux

codes suivants pour les processus P0 et P1 :

Alternance stricte

Processus P0

début
Répéter indéfiniment
TantQue Turn est différent de 0 faire
// attente active
FinTantQue
section critique ;
Turn = 1;
section non critique ;
FinRépéter
fin

processus P1

début
Répéter indéfiniment
TantQue Turn est différent de 1 faire
// attente active
FinTantQue
section critique ;
Turn = 0;
section non critique ;
FinRépéter
fin

algorithme de Peterson

- Peterson (1981) a trouvé une solution assez simple au problème de l'exclusion mutuelle. Deux variables globales sont utilisées : Turn, variable entière, et Desir[i], vecteur de valeurs booléennes (on ne considère ici que le cas de deux processus). L'algorithme de Peterson fait intervenir deux procédures :
- entrer_region() et quitter_region() :

algorithme de Peterson

entrer_region(numéro_de_processus)

Début

autre = 1 - numéro_de_processus ;

Desir[numéro_de_processus] = VRAI ;

Turn = autre ;

TantQue (Turn = autre et Desir[autre]=VRAI) faire

attendre ;

FinTantQue

fin

quitter_region(numéro_de_processus)

début

Desir[numéro de processus]=FAUX ;

fin

algorithme de Peterson

- Avant d'entrer dans leur région critique, chaque processus appelle entrer_region() en y passant comme paramètre son numéro de processus, 0 ou 1.
- Il peut alors entrer ou il doit attendre. A la sortie de la section critique, chaque processus doit invoquer quitter_region().

algorithme de Peterson

- Un inconvénient important également vient des règles d'ordonnancement. Supposons que l'on ait deux

processus H, bloqué et L, actif sur une machine monoprocesseur ; supposons que H possède une priorité plus forte que L et que les règles d'ordonnancement soient telles que si H est prêt, il doit devenir actif.

Supposons que L soit dans sa section critique ; à ce moment H passe de l'état bloqué à l'état prêt : il faut

que L passe à l'état prêt et H à l'état actif; mais H restera indéfiniment dans une attente active car L n'est pas sorti de sa section critique et L ne reviendra pas actif tant que H le sera !

Sleep et Wakeup

- Plutôt que des solutions avec attente active qui laissent les processus prêts, il faut rechercher des solutions qui bloquent les processus. Une des plus simples solutions est l'utilisation des appels systèmes

SLEEP et WAKEUP :

- SLEEP bloque le processus appelant jusqu'à son réveil par un autre processus
- WAKEUP réveille le processus dont le numéro lui est passé en paramètre

Producteur Consommateur

processus producteur :

```
début
 Répéter indéfiniment
    produire (item) ;
Si count = N alors sleep() ;
FinSi
entreposer (item) ;
count = count +1 ;
    Si count = 1 alors
wakeup(consommateur) ;
FinSi
FinRépéter
fin
```

processus consommateur :

```
début
Répéter indéfiniment
Si count = 0 alors
sleep() ;
FinSi
retirer(item) ;
count = count - 1 ;
Si count = N-1 alors
wakeup(producteur) ;
FinSi
consommer(item) ;
FinRépéter
fin
```

Producteur Consommateur

- Il y a une faille à ce système qui paraît séduisant . Supposons que le buffer soit vide et que le consommateur soit actif : il voit que count = 0, mais à ce moment, pour une raison quelconque, le dispatcher

lui enlève le processeur et le met en état prêt. Le producteur est, supposons, maintenant actif; il met une unité dans le buffer et croyant le consommateur endormi, envoie le signal de réveil avec wakeup; mais comme le consommateur ne dort pas, le signal est perdu. Lorsque le consommateur sera de nouveau actif, il s'endormira sous l'action de sleep car il croit toujours que le buffer est vide puisque, pour lui la dernière valeur lue de count est 0. Le producteur va alors continuer à remplir le buffer jusqu'à ce qu'il soit plein et s'endormira à son tour. Au bout du compte, on va avoir deux processus bien endormis pour toujours !

Producteur Consommateur

- On peut améliorer les choses en ajoutant un "wakeup waiting bit" : quand un signal wakeup est envoyé à un processus qui est déjà réveillé, le bit est positionné à 1. Plus tard, quand le processus doit dormir, le bit est remis à 0, mais le processus ne s'endort pas. Ce système semble marcher mais pour plus de deux processus cela devient plus difficile.

Sémaphores

- Un sémaphore est un outil de synchronisation . Il est matérialisé par une variable entière à laquelle est ajoutée une file d'attente de processus.
- Il n'est possible d'accéder à un sémaphore que par deux opérations notées WAIT et SIGNAL.
- Les primitives WAIT et SIGNAL sont définies comme suit

Sémaphores

s.WAIT : Début

s.val := s.val - 1

Si s.val < 0 alors

état(p) := bloqué

insérer p dans s.file

finsi

Fin

s.SIGNAL : Début

s.val := s.val + 1

Si s.val ≤ 0 alors

sortir un processus q de s.file

état(q) := prêt

finsi

Fin

Sémaphores

- Remarque :
- La valeur initiale du champ val d'un sémaphore doit être un nombre non négatif.

Sémaphores

Exemple : *Utilisation des sémaphores pour réaliser l'exclusion mutuelle*

Une ressource critique R est utilisée par un nombre quelconque de processus.

Processus Ayant_à_utiliser_R

début

<section non critique>

<utilisation de R>

<section restante>

fin

L'utilisation de R doit se faire en exclusion mutuelle pour tous les processus concernés.

Sémaphores

Pour réaliser l'accès en exclusion mutuelle pour l'accès à R , on utilisera un sémaphore initialisé à 1 :

var s : sémaphore init 1;

/* *s est une variable globale de type sémaphore* */

/* *dont la valeur est initialisée à 1* */

Processus Ayant_à_utiliser_R

début

<section non critique>

s.WAIT

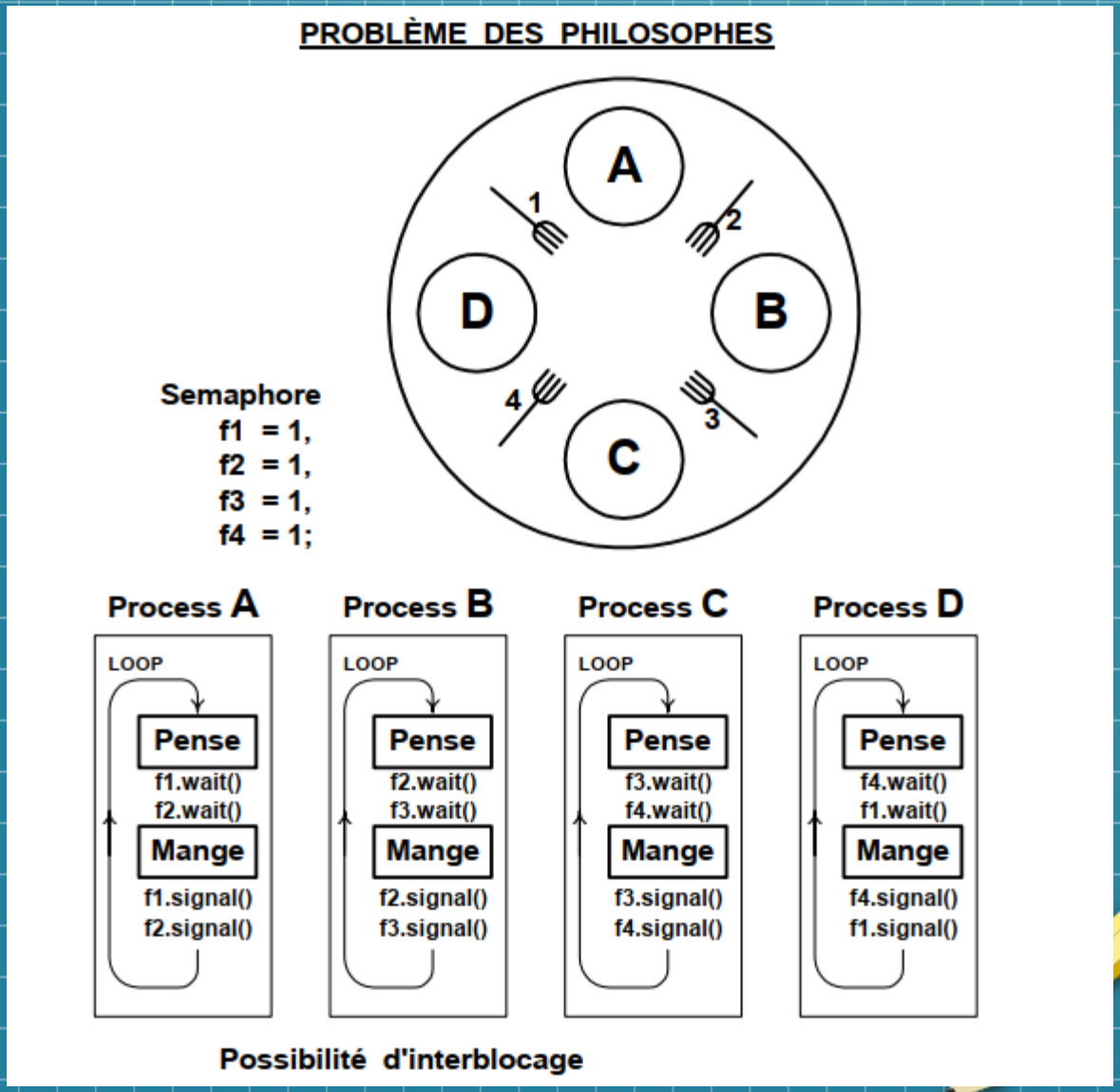
<utilisation de R> /* *section critique* */

s.SIGNAL

<section restante>

fin

Problème philosophe



Problème philosophe

- La solution de la Figure ci dessus montre qu'un philosophe doit faire un ***wait()*** sur chacune des 2 fourchettes qui l'entourent avant de manger. Il peut manger seulement s'il réussit à avoir les 2 fourchettes. Quand il a fini de manger, il relâche les 2 fourchettes avec un ***signal()***. Cette solution simple s'assure qu'un philosophe doit avoir 2 fourchettes pour manger et respecte l'interdépendance entre les processus. Cependant, elle peut produire un interblocage des processus. Supposons que tous les philosophes ont obtenu leur fourchette de droite (premier ***wait()*** dans les processus). Alors, aucun philosophe ne peut obtenir la fourchette de gauche
manquante. Les philosophes attendent mutuellement un après l'autre. Le processus A attend après B, B après C, C après D et D après A. Il y a attente circulaire. Nous avons un interblocage

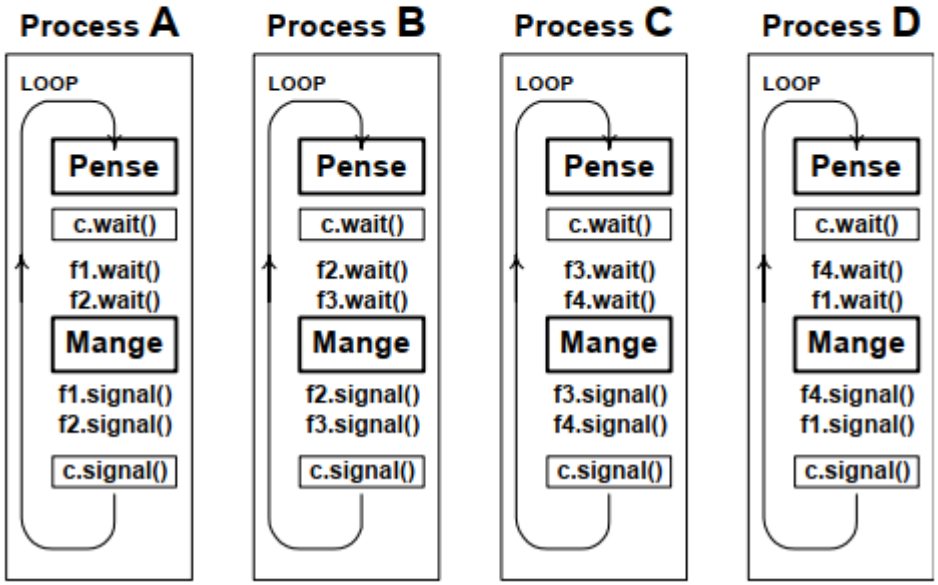
Problème philosophe

PROBLÈME DES PHILOSOPHES

Solution sans interblocage

```
Semaphore f1 = 1
          f2 = 1,
          f3 = 1,
          f4 = 1;

// autorise 3 philosophes seulement
Semaphore c = 3;
```



Problème philosophe

- La Figure ci dessus présente une solution sans interblocage. Elle consiste à s'assurer qu'au plus 3 philosophes utilisent des fourchettes simultanément. On évite ainsi l'attente circulaire. Les 3 processus qui ont des fourchettes ne pourront pas former une suite circulaire de processus qui s'attendent mutuellement. Il s'agit d'un problème d'exclusion mutuelle avec la modification que 3 processus peuvent exécuter le code "mange" en même temps. On utilise un sémaphore "c" qui est initialisé à 3. Tout au plus, 3 processus pourront faire le wait() sans bloquer.